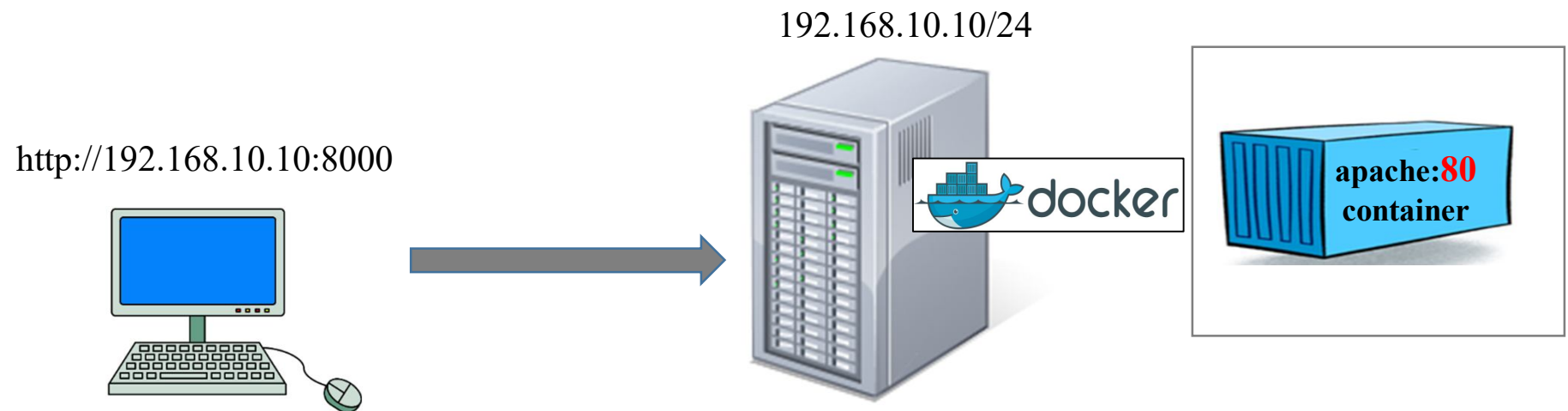


Docker Command

실습 0. Apache Web Server Container 생성과 실행



Container 접속 명령어

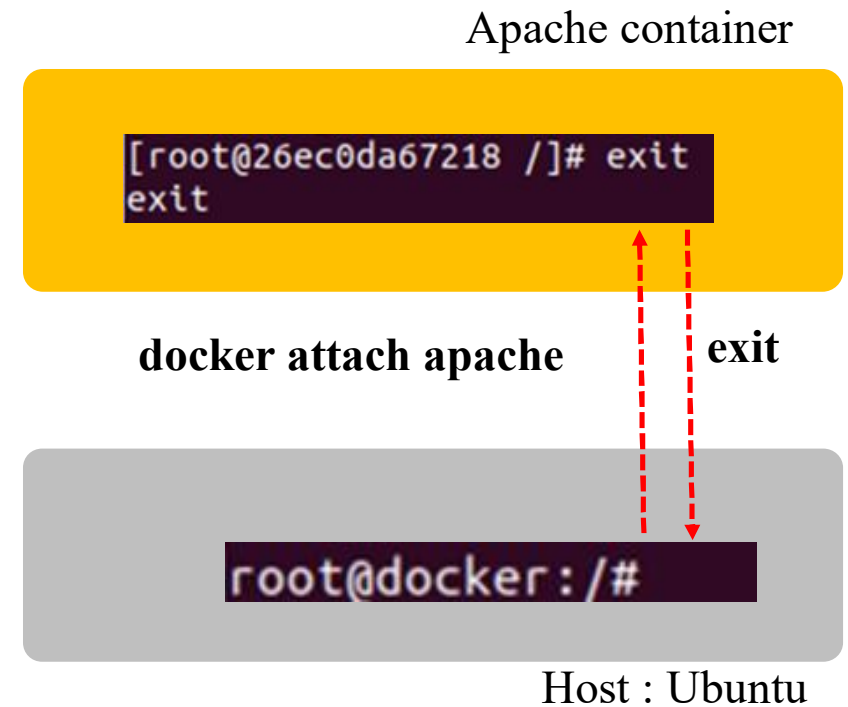
- docker exec 또는 docker attach
 - 컨테이너가 실행되고 있을 때 사용
 - docker ps 를 입력했을 때 실행 중인 컨테이너가 없다면 실행할 수 없음

docker exec <CONTAINER ID> <OPTIONS>	실행 중인 컨테이너에 명령어를 전달 (외부 -> 내부)
docker exec -it <CONTAINER ID> /bin/bash	실행 중인 컨테이너에 직접 들어가 명령어를 실행 (내부 접근)
docker run -it --name <CONTAINER NAME> /bin/bash docker attach <CONTAINER ID>	실행 중인 컨테이너에 직접 들어가 명령어를 실행 (내부 접근) */bin/bash로 컨테이너를 생성해야 attach로 접속 가능

docker attach로 Container 내부에 접근하여 명령어 실행

- ❶ `docker run -it --name apache -d httpd /bin/bash`
- ❷ `docker attach apache`
- ❸ `ls`
- ❹ `pwd`
- ❺ `exit`
- ❻ `docker start apache`
- ❼ `docker stop apache`
- ❽ `docker rm apache`
- ❾ `docker rmi apache`

***/bin/bash로 컨테이너를 생성해야 attach로 접속 가능**



docker exec를 이용한 Container에 명령어 전달

- ❶ `docker run --name apache -d httpd`
- ❷ `docker exec apache ls`
- ❸ `docker exec apache pwd`
- ❹ `docker exec apache cat /usr/local/apache2/htdocs/index.html`

`docker exec`로 Container 내부에 접근하여 명령어 실행

- ❶ `docker exec -it APS1 /bin/bash`
- ❷ `ls`
- ❸ `pwd`
- ❹ `cat /usr/local/apache2/htdocs/index.html`
- ❺ `exit`

Container 일시 정지와 재시작

- 구동중인 컨테이너에서 실행중인 프로세스를 모두 일시 정지 명령어 : `docker pause`

`#docker pause <컨테이너명 또는 ID>`

- 일시 정지된 프로세스 재시작 명령어 : `docker unpause`

`#docker unpause <컨테이너명 또는 ID>`

Container 정지와 재시작

- 컨테이너 정지 명령어 : docker stop

#docker stop <컨테이너명 또는 ID>

- 컨테이너 재시작 명령어 : docker restart

#docker restart [옵션] <컨테이너명 또는 ID>

항목	설명
-t	컨테이너 재시작 시간을 지정(default는 10초)

Docker 버전확인

- Docker 버전
- Go 언어버전
 - * 도커는 Go 언어^{Go}로 개발
- OS 확인
- 서버와 클라이언트정보확인

#docker version

```
root@ubuntu:/# docker version
Client:
 Version:           19.03.8
 API version:       1.40
 Go version:        go1.13.8
 Git commit:        afacb8b7f0
 Built:             Wed Mar 11 23:42:35 2020
 OS/Arch:           linux/amd64
 Experimental:      false

Server:
 Engine:
  Version:          19.03.8
  API version:      1.40 (minimum version 1.12)
  Go version:       go1.13.8
  Git commit:       afacb8b7f0
  Built:            Wed Mar 11 22:48:33 2020
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.3.3-0ubuntu2
  GitCommit:
 runc:
  Version:          spec: 1.0.1-dev
  GitCommit:
 docker-init:
  Version:          0.18.0
  GitCommit:
```

Docker 실행환경 확인

#docker info

```
root@ubuntu:/# docker info
Client:
  Debug Mode: false

Server:
  Containers: 1
    Running: 1
    Paused: 0
    Stopped: 0
  Images: 3
  Server Version: 19.03.8
  Storage Driver: overlay2
    Backing Filesystem: <unknown>
    Supports d_type: true
    Native Overlay Diff: true
  Logging Driver: json-file
  Cgroup Driver: cgroupfs
  Plugins:
    Volume: local
    Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-
  Swarm: inactive
  Runtimes: runc
  Default Runtime: runc
  Init Binary: docker-init
  containerd version:
  runc version:
```

컨테이너 수

```
init version:
Security Options:
  apparmor
  seccomp
    Profile: default
Kernel Version: 5.4.0-26-generic
Operating System: Ubuntu 20.04 LTS
OSType: linux
Architecture: x86_64
CPUs: 1
Total Memory: 1.914GiB
Name: ubuntu
ID: B15Q:T5JV:CNV5:JKCE:3LM3:2T7B:4R6V:
Docker Root Dir: /var/lib/docker
Debug Mode: false
Registry: https://index.docker.io/v1/
Labels:
Experimental: false
Insecure Registries:
  127.0.0.0/8
Live Restore Enabled: false

WARNING: No swap limit support
```

Docker호스트명

Docker저장소명

컨테이너 프로세스 확인

- 구동중인 컨테이너에서 실행 중인 프로세스 확인 확인 명령어 :

#docker top [옵션]

```
root@ubuntu:/# docker top e05c598255d6
```

UID	PID	PPID	C	STIME	TTY	TIME
-----	-----	------	---	-------	-----	------

Container 이름 변경

- 컨테이너 이름 변경 명령어 : `docker rename`

`#docker rename 이전컨테이너명 변경컨테이너명`

```
root@ubuntu:/# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
e05c598255d6       300e315adb2f      "/bin/bash"        4 minutes ago      Up 31 seconds        
compassionate_albattani

root@ubuntu:/# docker rename compassionate_albattani TEST02
root@ubuntu:/# docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
e05c598255d6       300e315adb2f      "/bin/bash"        4 minutes ago      Up 56 seconds        
TEST02
root@ubuntu:/#
```

#docker images

#docker ps -a

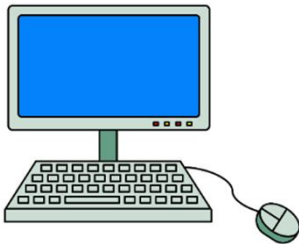
#docker rmi centos

#docker images

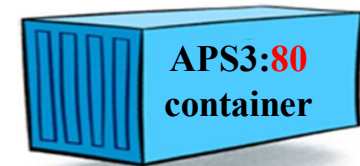
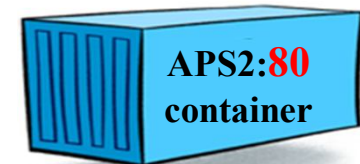
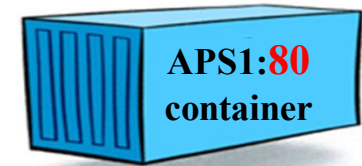
```
root@ubuntu:/# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
centos               latest             300e315adb2f       3 months ago       209MB
ubuntu              14.04             df043b4f0cf1       6 months ago       197MB
root@ubuntu:/# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
1a92731e3701       ubuntu:14.04       "/bin/bash"        2 hours ago        Exited (127) 2 hours ago
root@ubuntu:/# docker rmi centos
Untagged: centos:latest
Untagged: centos@sha256:5528e8b1b1719d34604c87e11dcd1c0a20bedf46e83b5632cdeac91b8c04efc1
Deleted: sha256:300e315adb2f96afe5f0b2780b87f28ae95231fe3bdd1e16b9ba606307728f55
Deleted: sha256:2653d992f4ef2bfd27f94db643815aa567240c37732cae1405ad1c1309ee9859
root@ubuntu:/# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
ubuntu              14.04             df043b4f0cf1       6 months ago       197MB
root@ubuntu:/#
```

실습 1. Apache Web Server Container 생성과 실행

http://192.168.10.10:8080
http://192.168.10.10:8081
http://192.168.10.10:8082



192.168.10.10/24



Container name	APS1	APS2	APS3
Image name	httpd	httpd	httpd
Port No.	8081:80	8082:80	8083:80

❶ `docker run --name APS1 -d -p 8081:80 httpd`

❷ `docker images`

`docker ps`

❸ `docker run --name APS2 -d -p 8082:80 httpd`

`docker run --name APS3 -d -p 8083:80 httpd`

❹ `docker images`

`docker ps`

`docker ps -a`

⑤ `docker exec -it APSX/bin/bash`

`cd /usr/local/apache2/htdocs`

`cat index.html`

`echo "<h1>Apache Server NO.X</h1>" > /usr/local/apache2/htdocs/index.html`

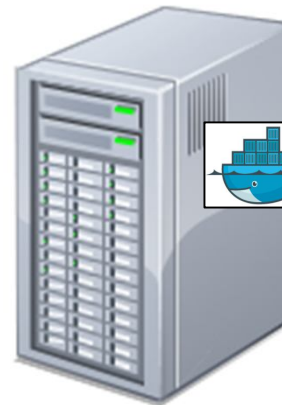
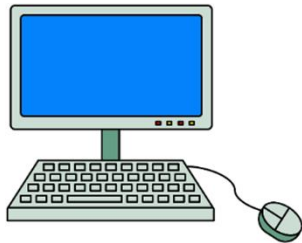
⑥ `http://192.168.10.10:8081`

`http://192.168.10.10:8082`

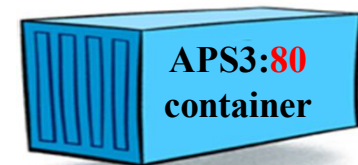
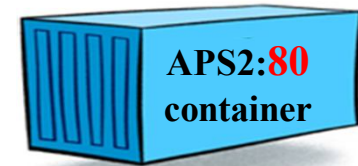
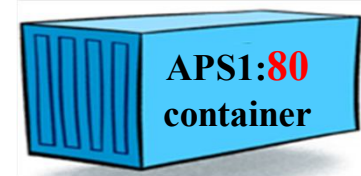
`http://192.168.10.10:8083`

실습 2. Nginx Web Server Container 생성과 실행

http://192.168.10.10:8084



Container name	NGX
Image name	nginx
Port No.	8084:80



❶ `docker run --name NGX -d -p 8084:80 nginx`

❷ `docker images`

`docker ps`

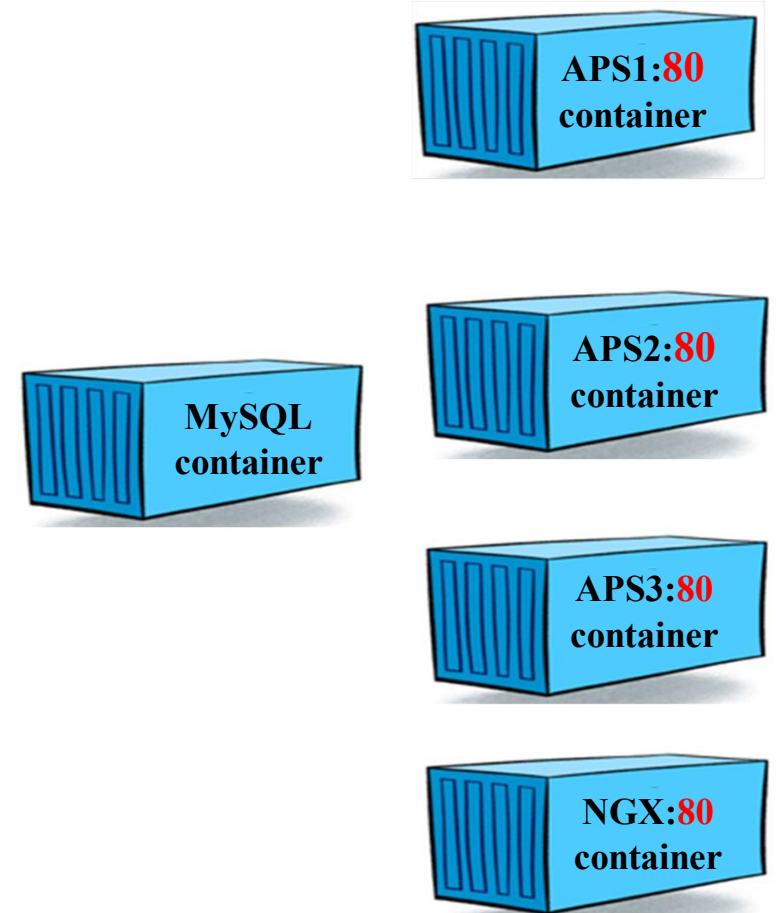
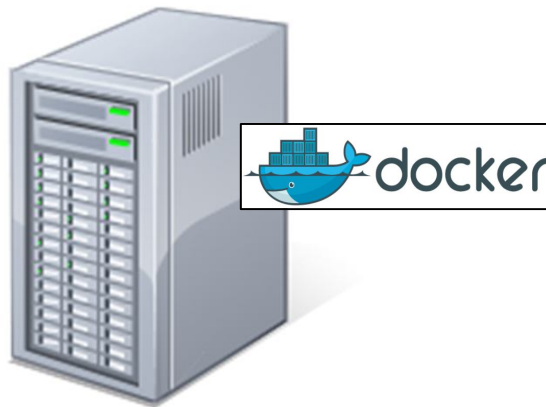
❸ `docker exec -it NXS /bin/bash`

`echo "<h1>Nginx Web Server </h1>" > /usr/share/nginx/html/index.html`

❹ `http://192.168.10.10:8084`

실습 3. MySQL Container 생성과 실행

Container name	MySQL
Image name	mysql
MySQL root Password	1234567



❶ `docker run --name MySQL --dit -e MYSQL_ROOT_PASSWORD=1234567 mysql`

❷ `docker images`

`docker ps`

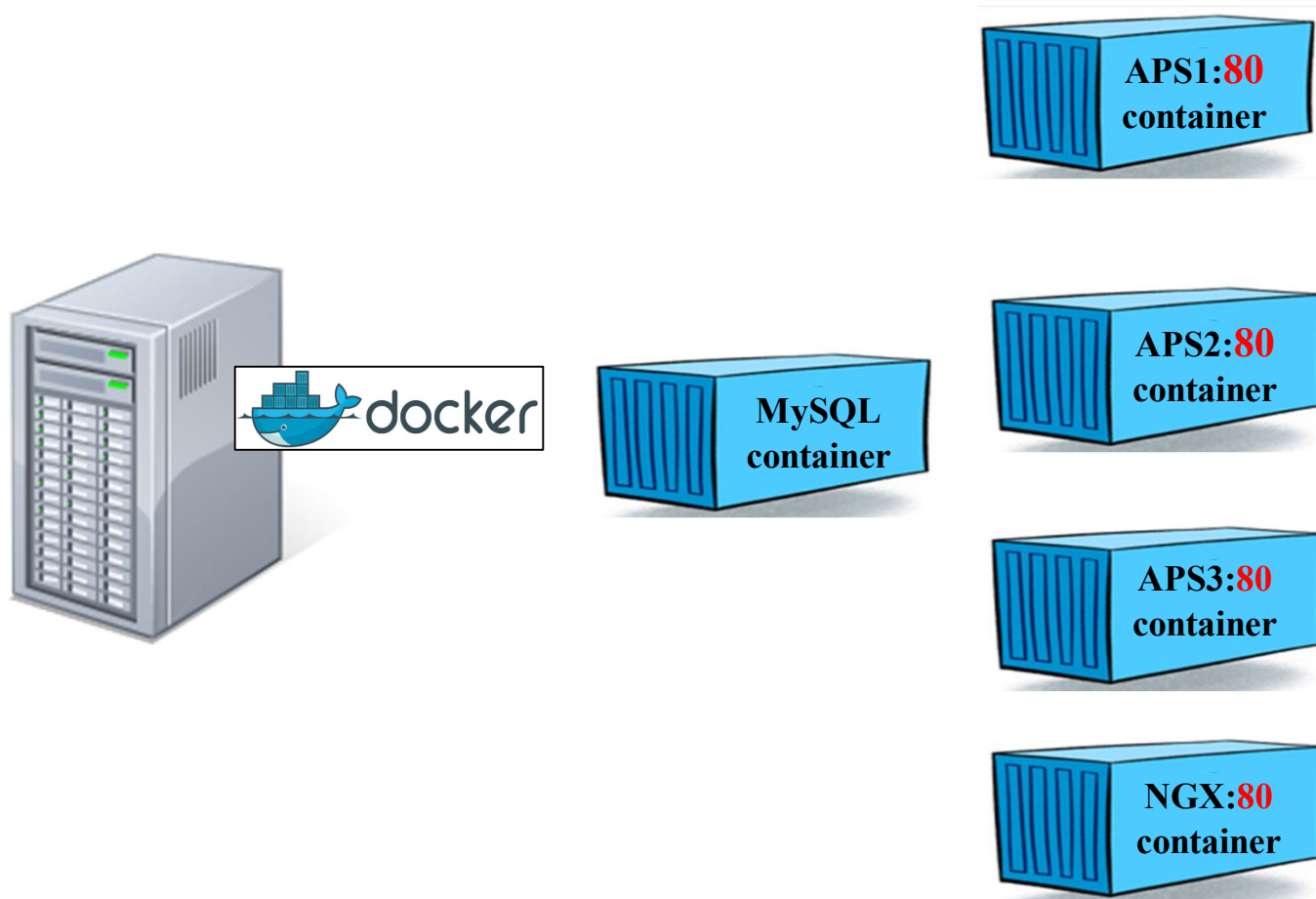
❸ `docker exec -it MySQL /bin/bash`

`mysql -u root -p`

`show databases;`

`exit`

실습 4. Container & Image 삭제하기



① docker stop *containername*

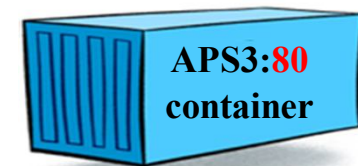
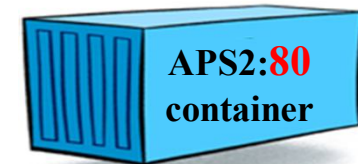
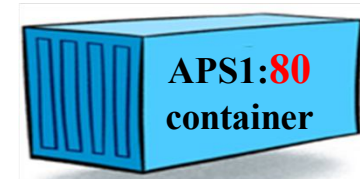
② docker ps

docker ps -a

③ docker rm *containername*

④ docker images

docker rmi *imagename*



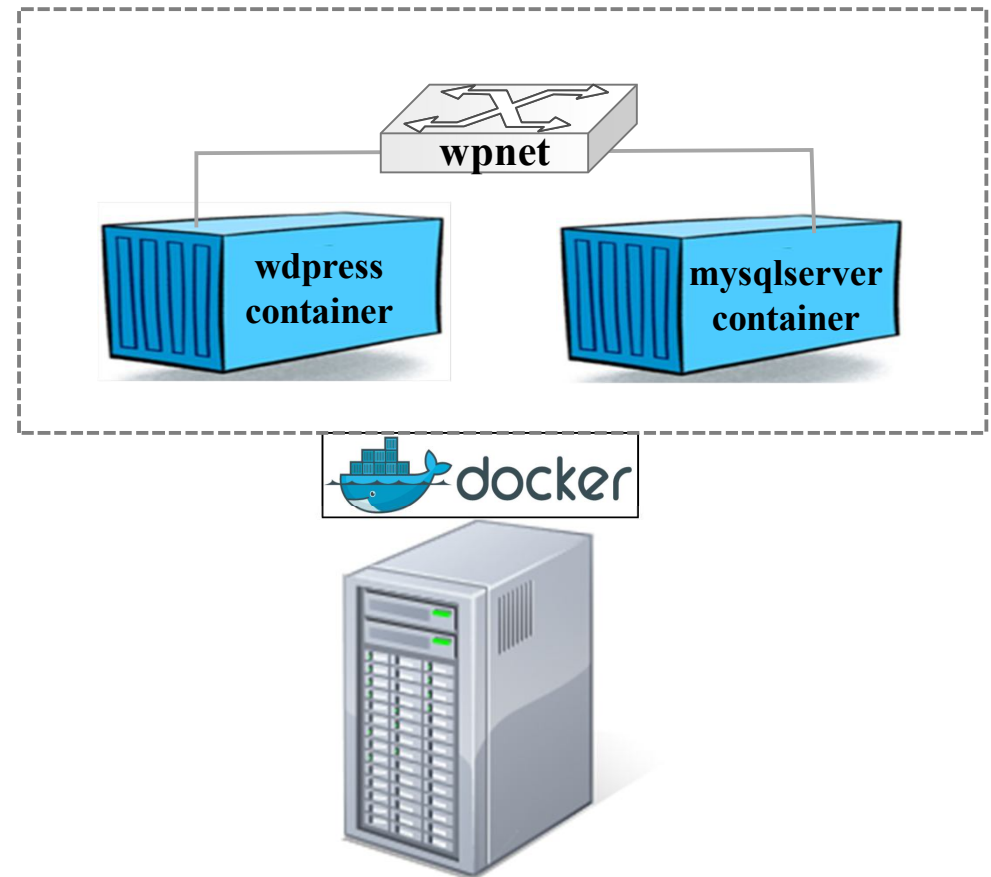
실습 5. WordPress Site 구축

<< Wordpress를 사용한 블로그 서버 구축 >>

- 네트워크 생성
- MySQL Container 생성
- WordPress Container 생성
- Container 및 네트워크 확인

* 네트워크 이름 : **wpnet**

	WordPress	MySQL
Container name	wdpress	mysqlserver
Image name	wordpress	mysql



① Network 생성

```
docker network create wpnet
```

[네트워크 생성 확인 명령어]

```
docker network ls
```

// 현재 호스트에 존재하는 모든 Docker 네트워크의 ID, 이름, 드라이버, 범위 출력

```
docker network inspect [네트워크_이름_또는_ID]
```

//네트워크가 생성된 것뿐만 아니라, 설정값 이나 연결된 컨테이너를 확인하고 싶을 때 사용

② MySQL Container 생성

```
#docker run --name mysqlserver --dit --net=wpnet
```

```
-e MYSQL_ROOT_PASSWORD=1234567
```

```
-e MYSQL_DATABASE=wordpressdb
```

```
-e MYSQL_USER=gildong
```

```
-e MYSQL_PASSWORD=1234
```

mysql

```
--character-set-server=utf8mb4
```

// 기본 문자 집합(Character Set)

```
--collation-server=utf8mb4_Unicode_ci
```

// 텍스트를 정렬, 비교할 때 사용할 규칙

```
#docker ps
```

③ WordPress Container 생성

```
docker run --name wdpres --dit --net=wpnet -p 8085:80
```

```
-e WORDPRESS_DB_HOST=mysqlserver
```

```
-e WORDPRESS_DB_NAME=wordpressdb
```

```
-e WORDPRESS_DB_USER=gildong
```

```
-e WORDPRESS_DB_PASSWORD=1234
```

wordpress

④ docker ps

⑤ http://192.168.10.10:8085



Basa Jawa
ქართული
Taqbaylit
Қазақ тілі
ភាសាខ្មែរ
ಕನ್ನಡ
한국어
کوردی
Кыргызча
ພາສາລາວ
Lietuvių kalba
Latviešu valoda
Македонски јазик
മലയാളം
Монгол
मराठी
Bahasa Melayu
මහලා
Norsk bokmål
नेपाली
Nederlands

계속

환영합니다

유명한 워드프레스 5분 설치 절차에 오신 것을 환영합니다! 아래의 정보를 입력하기만 하면 세계에서 가장 확장성 있고 강력한 개인 발행 플랫폼을 사용할 수 있습니다.

정보가 필요합니다

다음 정보를 제공해주세요. 걱정하지 마세요. 이 설정을 나중에 언제든지 바꿀 수 있습니다.

사이트 제목

Docker Blog

사용자명

Gildong

사용자명은 알파벳, 숫자, 스페이스, 밑줄, 하이픈, 마침표, @ 기호만 가능합니다.

비밀번호

bSJ09DxTcyTdQmIj!t

숨기기

강함

중요: 로그인할 비밀번호가 필요할 것입니다. 안전한 위치에서 저장해주세요.

이메일 주소

계속하기 전에 이메일 주소를 다시 확인하세요.

검색 엔진 가시성

☐ 검색 엔진이 이 사이트를 검색하는 것을 차단

이 요청이 받아들여지는 것은 전적으로 검색 엔진에 좌우됩니다.

워드프레스 설치



성공!

워드프레스가 설치되었습니다. 감사합니다. 즐거운 시간을!

사용자명 Gildong

비밀번호 *선택한 비밀번호*

[로그인](#)

실습 6. WordPress Site 삭제

❶ 컨테이너 삭제

```
docker stop wordpress
```

```
docker stop mysqlserver
```

```
docker rm wordpress
```

```
docker rm mysqlserver
```

❷ 이미지 삭제

```
docker image rm wordpress
```

```
docker image rm mysql
```

❸ 네트워크 삭제

```
docker network rm wpnet
```

* 네트워크 이름 : **wpnet**

	WordPress	MySQL
Container name	wordpress	mysqlserver
Image name	wordpress	mysql